Course: GENG 2290 – Software Fundamentals

Project: Arcade Claw Machine Simulator

Team #: 18

Members: Saja Chamas, Kabir Awasty,

Rohan Buch, Ethan Becker,

Ben Lanoue, Fares Ghanem

Date: Mar 2, 2026

1. Introduction

In this project, the design and development of an arcade claw machine simulator is given in C++. The purpose of the simulator is to assist an amusement company to test and fine tune the settings of machines prior to building an actual claw machine. The company can investigate how the settings influence the enjoyment of the players and machine profitability by testing various settings, including claw grip strength, price of the coins, the duration of play time and the availability of the prizes.

The simulator is based on the primary elements and functionality of a claw machine such as prizes, coin recognition (to start), statistics of every game, and access modes (play mode and operator mode). Simulation-wise, it will support both manual and automatic simulation to enable testing on a large number of trials.

This document combines the entire system design, problem statement, functional and non-functional requirements, class description, UML diagrams, algorithms, libraries and tools employed and task allocation between team members. The aim is to design a concise and well structured design that can be translated directly into C++ and satisfy project requirements and exhibit sound software engineering principles.

2. Problem Statement

An arcade company would like to make a claw machine which would be enjoyable to the players and at the same time profitable to the company. In case the claw grip is excessive, players win too much and the company makes losses. In case the claw grip is too feeble, players never win and they have no longer to play.

Before the company constructs a real machine, it has to find a method of testing the various machine settings. They desire to experience such things as claw strength, prize sizes, coin price and play time. They also should monitor some such important statistics like number of games played, number of wins, total revenue, payout percentage, and frequency of prizes restocking.

Our project addresses this issue by designing a C++ simulator of a claw machine. The simulator allows us to experiment with various settings and observe the outcome. This assists the owners of the arcades in establishing the optimal relationship between machine profit and player enjoyment.

3. System Requirements

3.1 Functional Requirements

The Statistics system should:

1. Track the total number of games played.
2. Track the total number of wins.
3. Track the total revenue from coins inserted.
4. Calculate the payout percentage based on wins and revenue.
5. Check if the machine is close to the 1 in 12 payout target.
6. Track how many prizes are left and estimate when restocking is needed.

7. Display all statistics to the operator when requested.

The Operator system should:

1. Allow the user to enter into operator mode to access various different functions to edit the game rules
2. Restock prizes
3. Change claw specifications
    a. Grip strength
    b. Recalibration
4. Collect revenue
5. Only display statistics while in operator mode

The Play Mode system should:

1. Allow user to enter coin and system will validate payment before activating joystick and drop button
2. Activate the joystick for a time limit once payment confirmed
3. Allow player to move claw using joystick
4. Descend the claw for prize attempt if:
    a. Player presses drop button
    b. Timer expires
5. Check if more than 200 plays have elapsed without re-service - if true it will apply a drift
6. Send game results to the Statistics class
7. Flag operator mode if the stock crosses assigned “restock needed” threshold

3.2 Non-Functional Requirements

The Statistics system should:

1. Update results immediately after each game.
2. Be accurate and reliable.
3. Show results clearly for the operator.
4. Work correctly with Play Mode and Operator Mode.
5. Run quickly without slowing the program.

The Prizes and grid system should:

1. Display where the claw and prizes are placed.
2. Display where the drop chute for prizes is.
3. Place the prize in a random location of the grid if a prize falls.

The Play Mode system should:

1. Track 30-second timer and terminate input when it expires (only if user input not detected)
2. Run automatic mode
3. Interface with ClawSpecifications, PrizeGrid, and CoinMechanism classes
4. Alert Statistics and Operator Mode classes

4. System Design

4.1 Statistics Class

The Claw machine class monitors the performance of the Claw machine.
Play Mode is the one that forwards the result of each game to this class. The operator will then be able to know whether the machine is generating or not, whether or not it is making money or losing money, and whether or not the machine needs more prizes.

The Statistics class tracks:
• gamesPlayed: total number of games played
• wins: number of prizes won
• revenue: total money collected from players
• payoutPercent: percentage of wins compared to games
• payoutRate: checks if the machine is close to 1 win every 12 games
• restockNeeded: checks if prizes are running low
Functions of the Statistics class:
• addGame(): increase games played by 1
• addWin(): increase wins by 1
• addRevenue(value): add coin value to total revenue
• calculatePayout(): compute payout percentage
• checkRestock(): check if prizes are low
• showStats(): display statistics to operator

This class works together with Play Mode and Operator Mode to help manage the claw machine.

4.2 Operator Class

The operator class is required to edit essential functions of the claw machine while operator mode is active.

Functions needed for operator class:

- enterOp(): Enters the user into operator mode, unlocking the service door and access to class functions
- clawGrip(float amount): Changes the grip setting on the claw from 10% to 100%
- clawFix(): Recalibrates claw if drifting or breakdown is observed (200+ plays)
- prizeRestock(int amount, int size): Restocks entered amount of prizes into machine, as well as specified size
- playLimit(float time): Changes the amount of time allowed each round after coin is inserted
- emptyCoins(): Collects all coins currently in the machine, returns none if none are present.
- exitOp(): Closes the service door and exits operator mode, restricting access to class functions. Reverts to default play mode

The operator class is dependent on only allowing edits to essential machine specifications if the service door is open, and operator mode is active. Otherwise any function called should not run when not in operator mode. This class works in tangent with statistics class and play mode, to transition between the default play mode, and operator mode.

4.3 Play Mode Class

The Play Mode class will manage the customer-side of the claw machine during each play session. It will run the simulation loop as demonstrated in the flowchart in 4.6. The attributes of this class are as follows:

- active: Flag whether play mode is active or not (bool)
- timer: Countdown for playtime before claw descends automatically (if button not pressed) (float)
- autoMode: Flag whether or not simulation is manual or automatic (bool)
- coinInserted: Read and track coin count to allow game access (float)

The operations (functions) of this class are as follows:

- startGame(): Validates coin input and starts joystick timer
- moveClaw(x, y): Move claw to the user-specified grid position
- dropClaw(): Descends the claw if the timer runs out or if the button is pressed. It applies drift if the condition applies. This will also call on grip calculation.
- endGame(): Resets claw to origin, sends results to Statistics class, alerts operator mode if restock needed, and sends check for next play(er)
- runAuto(): Random player runs and skill levels using <random> library
- insertCoin(coin): reads and accepts coin input to add to balance. Activates the game when $2.00 required balance is reached.

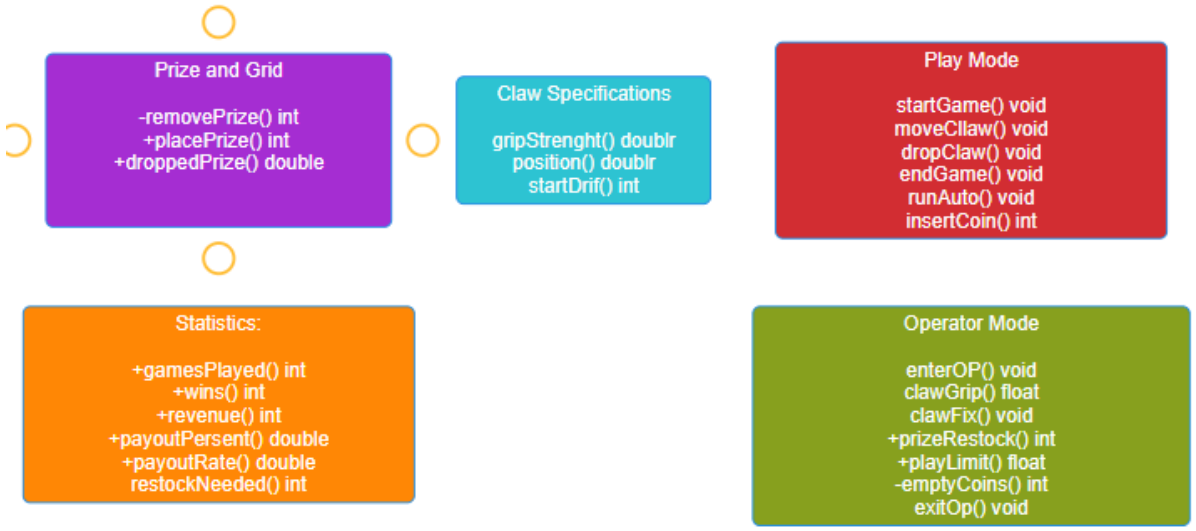
4.4 Prize and Grid Class

The prize and grid class is required in order to remove prizes when won and to add prizes to the game screen when the machine is restocked.

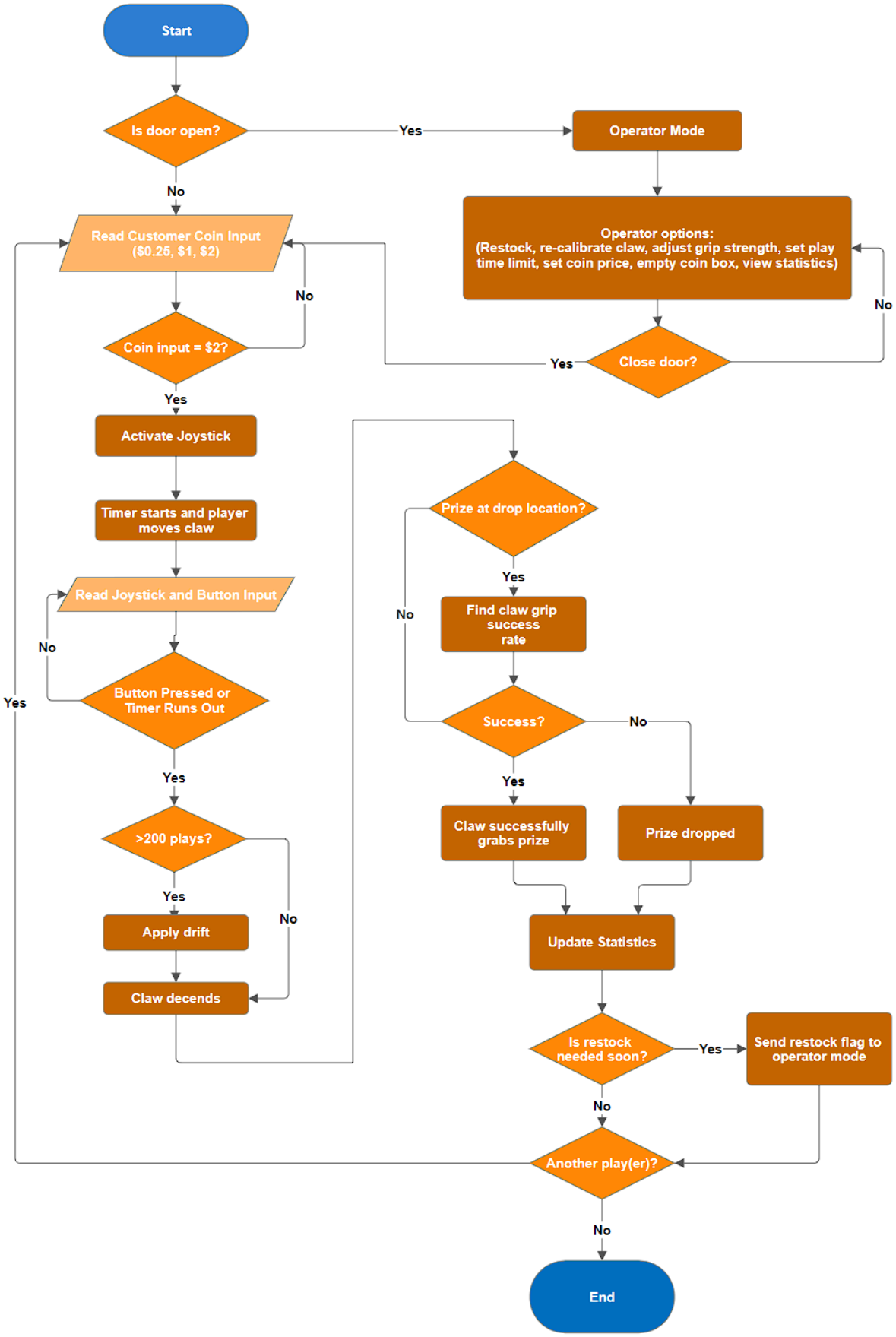
- removePrize(): Will remove the prize that was picked up by the claw if the prizes enter the prize shoot.
- placePrize(): Will take the value of the prizes added and will randomly place them into the game environment.
- droppedPrize(): Will place the prize randomly into any of the squares nearest to where the prize fell if not over the prize shoot.

This class will help to manage how the prizes are placed and removed from the claw machine and will help determine where a prize may fall if it slips from the claw during transit to the prize shoot.

4.5 UML Diagram



4.6 Flowchart



5. Algorithms

Algorithm 1 – Grab Success

For each game:

1. Move claw over to the prize that will be picked up
2. Run the function that will randomly generate the grip strength for that play
3. Using that grip strength number determines if the prize will be picked up or not
4. Using the same grip strength if high enough will pick up the prize and may or may not drop it
5. Return to the prize drop shoot with or without prize.

Algorithm 2 – Statistics Update

After each game:

1. Receive game result (win or lose)
2. Increase gamesPlayed by 1
3. Add coin value to revenue
4. If win → increase wins by 1
5. Calculate payoutPercent
6. Check if prizes are low → set restockNeeded
7. Send results to Operator Mode

Algorithm 3- Prizes and Grid

For each game:

1. Claw starts at the point (0,0)
2. Any prizes won removed from the grid
3. Large and small prizes labeled
4. Ability so see claw moving
5. If prize is dropped, locate nearest empty grid space and randomly place it there
6. Reset claw back to (0,0)

Algorithm 4 - Play Mode Loop:

For each player session:

1. Player inserts coins - add each input to balance
2. If balance is less than $2, loop and prompt for more input
3. Once balance = $2, activate joystick and start timer
4. Read user input for joystick and update on grid while timer is still running
5. If drop button pressed or timer expires, check if elapsed plays without service equals to or exceeds 200
6. If it exceeds 200, apply drift and offset claw position
7. Claw descends, call Grab Success algorithm for outcome

8. Outcome returned to Play Mode (if win or lose)
9. Call Statistics Update algorithm with game result
10. Call Prizes and Grid Algorithm to update grid and reset claw
11. If in auto mode, repeat from step 1 with new player
12. If in manual mode, prompt for next play(er) and repeat or end simulation

6. Libraries and Tools

The libraries that we intend to use for this project are:

1. <raylib.h> , this library will allow for the game environment to be created and allows for many game features to be used
2. <random>, this will help to simulate the probability of many of the occurrences in the project such as, grip chance success, the prize slipping from the claw, the drift after 200 plays and much more.
3. <chrono>, this library will be used for the timing of when the joystick should be active
4. <queue>, this library will allow for the program to determine at what time players arrive at the machine.

The tools that will be used for this project include:

1. Visual Studio or Visual Studio code, these platforms will be the main areas used when developing, debugging and testing the program.
2. GitHub, GitHub will be used as a storage location for the project and as a project bored as all files will be stored there so every member can access them, and a list of running tasks that need to be completed.
3. SmartDraw, SmartDraw will be used for any diagrams that the group will be required to make for papers or presentations.

7. Task Distribution

The task distribution was broken down based on different sections required to make the project functional.

1. **Claw Specs - Kabir:** in charge of grip functions, claw positioning, and claw breakdown features.
2. **Prizes and Grid - Ben:** designing grid system for prizes, prize sizes, drop chute positioning, and success and failure conditions for grabbing a prize.
3. **Statistics - Saja:** tracking games played, winrates, payout, and how often restocking is needed.
4. **Operator mode - Ethan & Fares:** mode to edit claw specs, coin prices and collect deposited coins, refill prizes, recalibrate claw after breakdown as well as view stats
5. **Play mode - Rohan:** make machine interactive or automatic, time limit implementation while playing, controls for machine and algorithm for random players while automatic

8. Conclusion

The proposed arcade claw machine simulator will provide a C++ solution to model player engagement with profitability without the company having to build a claw machine and physically testing it. By adding real time statistical adders the company will be able to monitor the amount of games played on their machine, the number of wins out of those games, the revenue and when the machine needs to be restocked. The operator class will allow real time adjustments in the simulation which allow for the claw to be recalibrated after 200 games played, be able to change the amount of time after coins are inserted and be able to empty out the coin box and bill acceptor. The automatic simulation assists the business in testing settings thousands of times and in normal play. We can simulate various skills of players and arrival rates with the help of the queue and random libraries to maintain the payout ratio of 1-in-12. The simulator also provides an elaborate financial prediction ahead of any actual manufacturing due to the 200 play recalibration cycle, on the grid of 10x8. This approach fills the differences between the hypothetical machine environments and actual arcade incomes.